ARQUITECTURA MVC

Decorator, Proxy y Bridge

Juan Martinez
Jeison Diaz
Samuel Clavijo

ADSO 3315796

QUE ES Y PARA QUE SIRVE

El Modelo-Vista-Controlador (MVC) es un patrón de arquitectura de software que divide una aplicación en tres componentes interconectados.

Su función es separar las diferentes responsabilidades de la aplicación, lo que permite desarrollar aplicaciones complejas, mantenibles, escalables y de fácil expansión

El patrón se organiza en tres capas independientes:

- Modelo (Model): Gestiona la lógica de datos, reglas del negocio, validaciones y la comunicación con bases de datos o API. Es independiente de la interfaz de usuario
- Vista (View): Representa la interfaz gráfica (GUI/Frontend). Decide qué verá el usuario en pantalla y cómo se estructuran y presentan los datos
- Controlador (Controller): Es el cerebro e intermediario. Recibe la petición del usuario desde la vista, solicita o modifica información en el modelo y selecciona la vista que debe renderizarse como respuesta

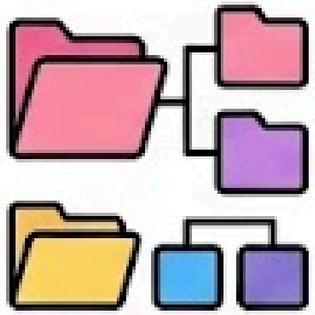
CARACTERÍSTICAS

1. División en modelo, vista y controlador.
2. Cada componente tiene una responsabilidad específica.
3. El controlador coordina la comunicación entre modelo y vista.
4. El usuario interactúa con la vista.
5. El modelo gestiona los datos y la lógica de la aplicación.
6. Permite mantener una estructura organizada y modular.

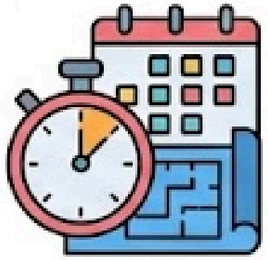


VENTAJAS Y DESVENTAJAS

- Permite separar las diferentes responsabilidades de la aplicación y mantener el código organizado.
- Facilita modificar una parte del proyecto sin afectar directamente las demás
- Permite reutilizar componentes en diferentes partes del proyecto
- Facilita el trabajo en equipo al permitir que varios desarrolladores trabajen en distintos componentes
- Permite agregar nuevas funcionalidades fácilmente a medida que la aplicación crece

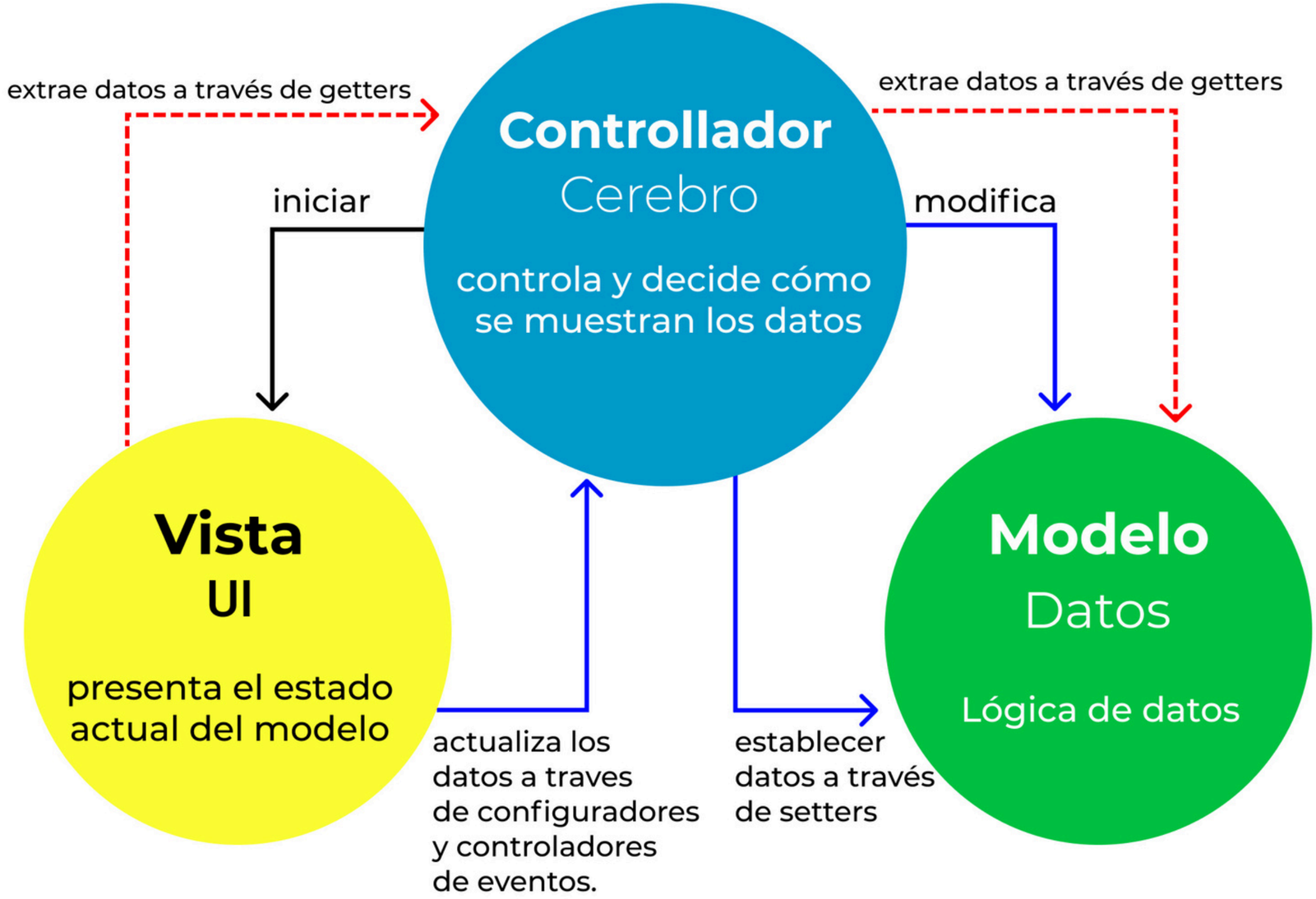


- Puede generar una mayor cantidad de archivos al separar las responsabilidades de la aplicación
- Puede ser innecesario para proyectos pequeños
- Requiere una buena organización para evitar dificultades en la comunicación entre los componentes

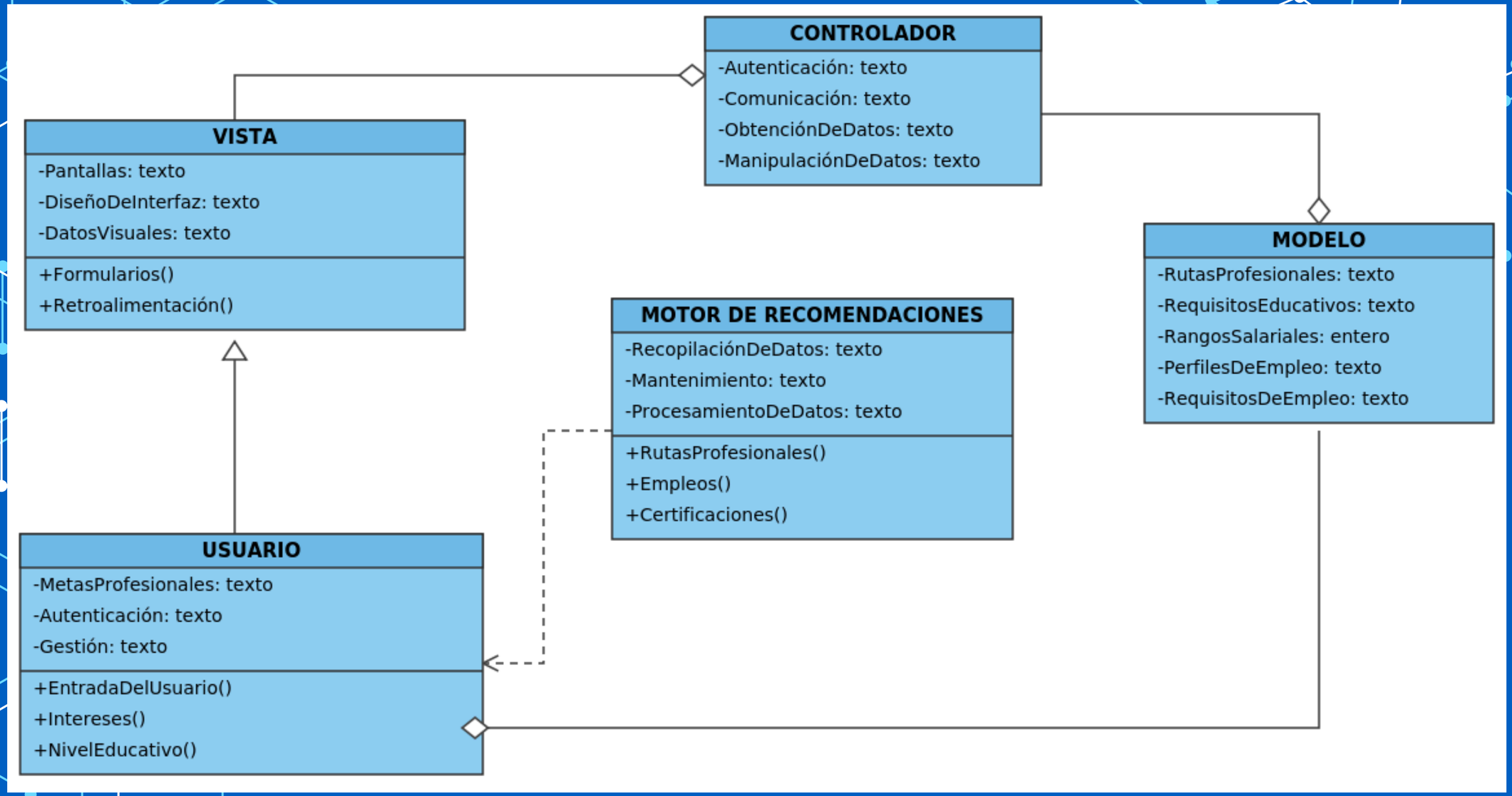


DIAGRAMA

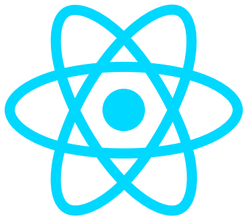
Patrones de Arquitectura MVC



DIAGRAMA


CONTROLADOR
-Autenticación: texto
-Comunicación: texto
-ObtenciónDeDatos: texto
-ManipulaciónDeDatos: texto
VISTA
-Pantallas: texto
-DiseñoDeInterfaz: texto
-DatosVisuales: texto
+Formularios()
+Retroalimentación()
MODELO
-RutasProfesionales: texto
-RequisitosEducativos: texto
-RangosSalariales: entero
-PerfilesDeEmpleo: texto
-RequisitosDeEmpleo: texto
MOTOR DE RECOMENDACIONES
-RecopilaciónDeDatos: texto
-Mantenimiento: texto
-ProcesamientoDeDatos: texto
+RutasProfesionales()
+Empleos()
+Certificaciones()
USUARIO
-MetasProfesionales: texto
-Autenticación: texto
-Gestión: texto
+EntradaDelUsuario()
+Intereses()
+NivelEducativo()

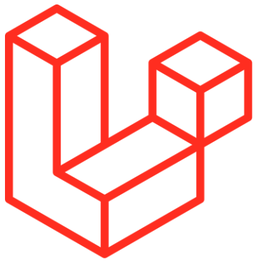
TECNOLOGIAS RELACIONADAS



REACT



NODE.JS



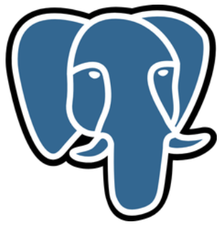
LARAVEL



SPRING MVC



MYSQL



POSTGRE SQL

conclusion

MVC separa la aplicación en Modelo, Vista y Controlador. Eso hace el código más ordenado, fácil de mantener y de probar, y permite cambiar la interfaz sin tocar la lógica de datos.

Conviene usarlo cuando:

- La aplicación es mediana o grande y va a seguir creciendo.
- Trabajan varios desarrolladores a la vez
- Necesitas varias vistas de los mismos datos, por ejemplo web y móvil.
- Quieres hacer pruebas por separado de la lógica y de la interfaz.

No conviene cuando:

- Es un programa muy pequeño o un script simple.
- Es un prototipo rápido, donde tantos archivos solo complican el trabajo

Patrones de diseño: Decorator, Proxy y Bridge

QUE SON PATRONES DE DISEÑO

Son soluciones reutilizables y probadas para problemas comunes que ocurren durante el diseño y desarrollo de software.

Los cuales funcionan para

- Acelerar el desarrollo: Proporcionan paradigmas de diseño probados que evitan que tengas que "reinventar la rueda".
- Estandarizar el código: Crean un vocabulario común entre desarrolladores. Decir "usémos un Singleton" o "hagamos un Factory" comunica instantáneamente la intención y la estructura sin necesidad de explicaciones extensas.
- Mantenibilidad y Escalabilidad: Promueven principios como el desacoplamiento, la flexibilidad y la facilidad para realizar pruebas automáticas

CARACTERISTICAS

DECORATOR:
Permite añadir nuevas funcionalidades a un objeto sin modificar su clase original.
Utiliza objetos que envuelven al objeto original para ampliar sus funcionalidades.
Usa varios decoradores para agregar diferentes características a un mismo objeto.

PROXY:
Actúa como intermediario entre el cliente y el objeto real para controlar la comunicación.
Verifica permisos o condiciones antes de permitir que el cliente acceda al objeto Controla y
Limita el acceso al objeto real según sea necesario.

BRIDGE:
Permite que una misma función trabaje con diferentes implementaciones.
Puede cambiar una clase sin tener que modificar las demas.

VENTAJAS Y DESVENTAJAS

DECORATOR:

Ventajas:
Evita crear muchas clases, permite combinar funciones, facilita la reutilización del código.
Desventajas:
Puede aumentar la cantidad de objetos.

PROXY:

Ventajas:
Controla el acceso a los recursos, mejora la seguridad, permite controlar recursos.
Desventajas:
Puede hacer más lento el acceso.

BRIDGE

Ventajas:
Modifica una parte sin afectar directamente a la otra, evita código repetido, facilita el mantenimiento del codigo.
Desventajas:
Aumenta la cantidad de clases.

DEMOSTRACION-CODIGO DE SIMULACION

Modelo (model)

```
// userModel.js
const users = [
  { id: 1, name: "Ana López", role: "Desarrolladora" },
  { id: 2, name: "Carlos Gómez", role: "Diseñador" },
  { id: 3, name: "María Rodríguez", role: "Project Manager" }
];

class UserModel {
  static getAllUsers() {
    return users;
  }

  static getUserById(id) {
    return users.find(u => u.id === Number(id));
  }
}S

module.exports = UserModel;
```

Vista (view)

```
class UserView {
  static renderUsersList(users) {
    const items = users
      .map(u => `<li><a href="/user?id=${u.id}">${u.name}</a> - ${u.role}</li>`)
      .join("");

    return `
      <!DOCTYPE html>
      <html lang="es">
      <head><title>Lista de Usuarios</title></head>
      <body>
        <h1>Lista de Usuarios</h1>
        <ul>${items}</ul>
      </body>
      </html>
    `;
  }

  static renderUserDetail(user) {
    if (!user) {
      return `<h1>Error 404</h1><p>Usuario no encontrado.</p><a href="/">Volver</a>`;
    }

    return `
      <!DOCTYPE html>
      <html lang="es">
      <head><title>Detalle de Usuario</title></head>
      <body>
        <h1>Perfil de ${user.name}</h1>
        <p><strong>ID:</strong> ${user.id}</p>
        <p><strong>Rol:</strong> ${user.role}</p>
        <a href="/">← Volver a la lista</a>
      </body>
      </html>
    `;
  }
}
module.exports = UserView;
```

Controlador (controller)

```
// userController.js
const UserModel = require('./userModel.js');
const UserView = require('./userView.js');

class UserController {
    static handleHome(req, res) {
        const users = UserModel.getAllUsers();
        const html = UserView.renderUsersList(users);

        res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
        res.end(html);
    }

    static handleUserDetail(req, res, id) {
        const user = UserModel.getUserById(id);
        const html = UserView.renderUserDetail(user);

        res.writeHead(user ? 200 : 404, { 'Content-Type': 'text/html; charset=utf-8' });
        res.end(html);
    }
}

module.exports = UserController;
```

GRACIAS